Scaling Bayesian Optimization via Thompson Sampling Using Derivative Information

Seong Woo Han*
University of Pennsylvania
Pennsylvania, PA 19104
seonghan@seas.upenn.edu

Abstract

Bayesian Optimization is a popular method for the sample-efficient optimization of expensive black-box functions. Bayesian Optimization uses an acquisition function that proposes sampling points in the search space where an improvement over the current best observation is likely. Thompson sampling is a well-known acquisition function that addresses the exploration-exploitation dilemma. However, the application of Thompson sampling as an acquisition function in high-dimensional Bayesian Optimization problems remains challenging due to the curse of dimensionality. In this paper, we propose a Thompson sampling method using derivative information that does not have to evaluate all grid points in the search space of high-dimension problems. We show using derivative methods drastically shortens the time to propose the sampling points in the high-dimensional space without investigating all grid points.

1 Introduction

Bayesian optimization (BO) has emerged as an effective, sample-efficient approach for optimizing a black-box function consisting of a probabilistic surrogate model and an acquisition function that leverages the surrogate model to propose a set of new candidate points to be evaluated [11]. A Gaussian Process (GP) is a popular model used as the surrogate model because it is a flexible non-parametric model that is known for its well-calibrated and posterior uncertainty estimates [16]. The acquisition function is responsible for balancing exploration and exploitation. Popular acquisition functions include expected improvement [11], entropy based information theory method, [9] and Thompson sampling [15].

While BO has emerged as a highly competitive tool for problems with a few tunable parameters [6, 17], it often scales poorly to high dimensions and large sample budgets. High-dimensional optimization is hard for a couple of reasons. First, the search space increases exponentially as the dimension increases. Second, since the search spaces grow faster than sampling budget due to the curse of dimensionality, this results in large posterior uncertainty region, which gives an overemphasized exploration and challenging to exploit promising search space.

To overcome these challenges, we adopt derivative information in Thompson sampling to optimize the generated sample faster. First, we compare three derivative methods—vanilla gradient descent, gradient descent with momentum, and L-BFGS—applied to Thompson sampling and show how many numbers of model evaluation it takes to propose the next sampling point as the dimension increases. Second, we test how this method finds the local minimum in GP trained cosine and convex functions. Finally, we apply this Thompson sampling using derivative information as an acquisition function in Bayesian optimization on optimization function to test its efficiency.

*Use footnote for providing further information about author (webpage, alternative address)—*not* for acknowledging funding agencies.

Preprint. Under review.

1.1 Related Work

BO has recently become the premier technique for global optimization of expensive functions, with applications in hyperparameter tuning, drug discovery, and materials discovery. However, BO is subject to important limitations in its scalability. In particular, its successful applications typically consider low-dimensional search spaces [12, 21, 14].

There have been many attempts to scale BO to more dimensions and observations by replacing the GP model: [18] uses Bayesian linear regression from neural networks, whereas [10] uses random forests. There are many other methods that use additive structure [13, 7, 20]. However, both low-dimensional linear structure and additive structure need strong assumption, and these methods often perform poorly if the assumed structure does not exist [3]. This becomes problematic when optimizing multiple objective because all objectives need to have the same assumed structure, which is unlikely in practice. The above methods typically rely on global GP models, which is also problematic: (i) as the dimensionality of the search space grows, the boundary issue discussed becomes more prominent; (ii) a global GP model scales poorly with the number of function evaluations.

High-dimensional problems with large sample budget have been studied extensively in simulation optimization and operation research [2]. Here, the trust region methods are based on a local surrogate model in a region (often a sphere) around the best solution. [4] proposes TuRBO, a high-dimensional BO that avoids over-exploration by performing optimization in local trust regions. TuRBO is designed for the large evaluation setting with large batches sizes and makes no strong assumption on the objective function. SCBO [5] extends TuRBO to handle black-box constraints, but this approach is still limited to single-objective optimization. Both TuRBO and SCBO fit GP models to the entire history of data collected by a single trust region even though the optimization is restricted to a local trust region.

2 Thompson Sampling using Derivative information

In this section, we provide details and pseudocode on Thompson sampling using derivative information with GP precise. Thompson sampling is an effective stochastic policy for the multi-armed bandit problem that maintain a belief about which option is optimal in given information. It designs each evaluation by sampling from this distribution, which addresses the exploration-exploitation dilemma by sampling a point proportional to its probability of maximizing the objective.

The key point in Thompson sampling is the posterior distribution of the location of the global maximum x^*, $p(x^*|D)$, a distribution with a given data D. Thompson sampling chooses the next observation location by drawing a sample from this belief, yielding a nondeterministic optimization policy:

$$x \sim p(x^*|D). \tag{1}$$

The location of the global maximum x^* is determined by the objective function f. We first sample a random realization of the objective function from its posterior $p(f|D)$:

$$\alpha_{TS}(x; D) \sim p(f|D). \tag{2}$$

Then, we optimize the function to the achieve the desired sample from $p(x^*|D)$, the next observation location:

$$x \in \operatorname*{argmax}_{x' \in X} \alpha_{TS}(x'; D). \tag{3}$$

We can interpret the sampled objective function α_{TS} as an acquisition function that is maximized. This procedure yields a sample from the joint distribution of the location and value of the optimum, $p(x^*, f^*|D)$, as the value of the sampled objective function α_{TS} at its maximum provides a sample from $p(f^*|x^*, D)$.

The method we chose is to maximize a draw from the objective function posterior, built continuously on demand using gradient information. If an optimization routine requests an evaluation of α_{TS} at a point x, we sample an objective function value conditioning on that point:

$$\phi \sim p(\phi|x, D), \tag{4}$$

In each iteration, the gradient information is used to optimize the generated sample by sampling from the joint posterior of the function value and its gradient:

$$p(\phi, \nabla\phi|x, D_{TS}), \tag{5}$$

This process augments our dataset with the simulated observation (x, ϕ) and is proceeded until the optimizer terminates. The Pseudocode is provided in Algorithm 1 in the form of a generator function - the "yield" statement returns the value, while maintaining the state.

Algorithm 1 Thompson sampling with derivative information

```
D_TS ← D                                  ▷ initialize a fictious dataset with current data
while until external optimizer terminates do
    given request for observation at x:
    φ ← p(φ, ∇φ|x, D_TS)                  ▷ sample value a x
    D_TS ← D_TS ∪ (x, φ)                  ▷ update a fictious dataset
    yield φ
```

Conceptually, Thompson sampling [19] for BO works by drawing a function f from the surrogate model (GP) posterior. Then, it makes a suggestion by reporting the optimum of the function f. This process is repeated independently for multiple suggestions, which naturally handles the exploration-exploitation trade off. However, in high dimensions, the additional cost required by this approach become excessive because in d dimension, a gradient observation increase the number of values we condition on by a factor of $(d + 1)$. In section 3.1, we compare the three derivative information we apply in Thompson sampling. Using those methods, we sample from the GP and increase the dimensions to observe the relationship between the model evaluations and dimension.

3 Experiments

3.1 Derivative information comparison

We first apply and compare three derivative information in Thompson sampling. First-order optimizers, vanilla gradient descent and gradient descent with momentum, are considered, and the second-order optimizer L-BFGS is considered [22]. Second-order optimizer such as L-BFGS have been preferred for optimizing GP hyperparameters [16], but they tend to perform poorly in the presence of noisy estimates of gradients [8, 1]. However, among these three methods, we found using BFGS in Thompson sampling excels the other two methods in suggesting next sample points. For each dimension, we repeated the experiment 100 times to achieve an average number of model evaluation.

In Figure 1, we observe the number of model evaluations for vanilla gradient descent increases exponentially as the dimension increases. We can also observe the model evaluation for gradient descent with momentum increases exponentially when compred with L-BFGS. The learning rate of 0.5 is used for vanilla gradient descent, and the momentum of 0.6 is used for the gradient descent with momentum.

The range of x label is only set up to 8 dimension when comparing three because the model evaluation for vanilla gradient descent and gradient descent with momentum increase exponentially after 8 dimension. Figure 2 shows Thompson sampling using L-BFGS takes an average number of 41 model evaluations up to 100 dimension.

3.2 Using Derivative Thompson sampling from GP function

In this section, we test derivative Thompson sampling method in a GP trained convex function and a cosine function to evaluate its efficiency. For each function, we first train the functions in GP and observe where the sampling points converge at last from the initial sampled points in the function.

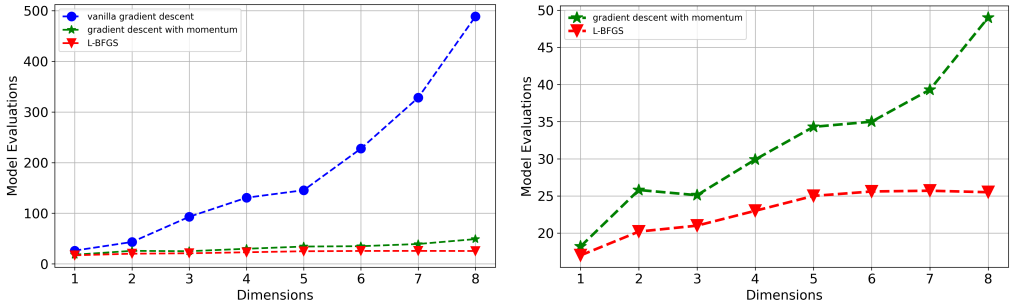

Figure 1: A comparison of vanilla gradient descent, gradient descent with momentum, and L-BFGS applied in Thompson sampling: the left panel compares the three methods as the dimensionality is varied. The right panel only compares gradient descent with momentum and L-BFGS to closely observe their efficiency as the dimensionality is varied. For each dimensionality, the experiment is conducted 100 times to achieve an average number of model evaluations.

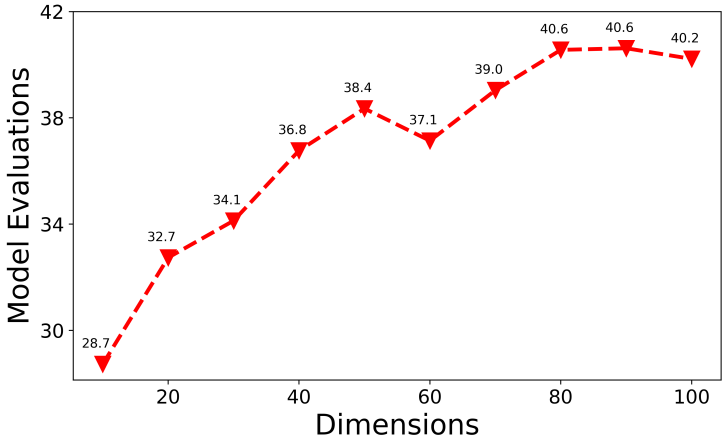

Figure 2: Thompson sampling using L-BFGS in 100 dimension. For each dimensionality, the experiment is conducted 100 times to achieve an average number of model evaluations.

The last point should converge nearby the local minimum, where the derivative of the slope is 0. Figure 3 shows the process of how derivative Thompson sampling finds the nearest local minimum in GP trained convex function from its initial sampled point. The orange points are the proposed points done by line search using L-BFGS, and the red points are the best suggested point among the proposed points. The last panel shows the trajectory of how Thompson sampling using L-BFGS finds the local minimum from a convex function.

We also do the same process for cosine function. For cosine function, we only the final panel that shows the trajectory of how Thompson sampling using L-BFGS finds the local minimum from a GP trained cosine function in the left panel of Figure 4 to see the same process.

Thompson sampling with L-BFGS finds the correct local minimum in these two cases; however, when repeating each experiment 100 times and observe Histogram 5 where the final points end up, we can also see the final location ends up in -9 in cosine function. In this case, we can observe L-BFGS overshoots far from the nearest local minimum in the right panel of Figure 4.

Using L-BFGS is fast in terms of suggesting the next sample point in high dimension, but it tends to perform poorly when it does not know when to stop line search.

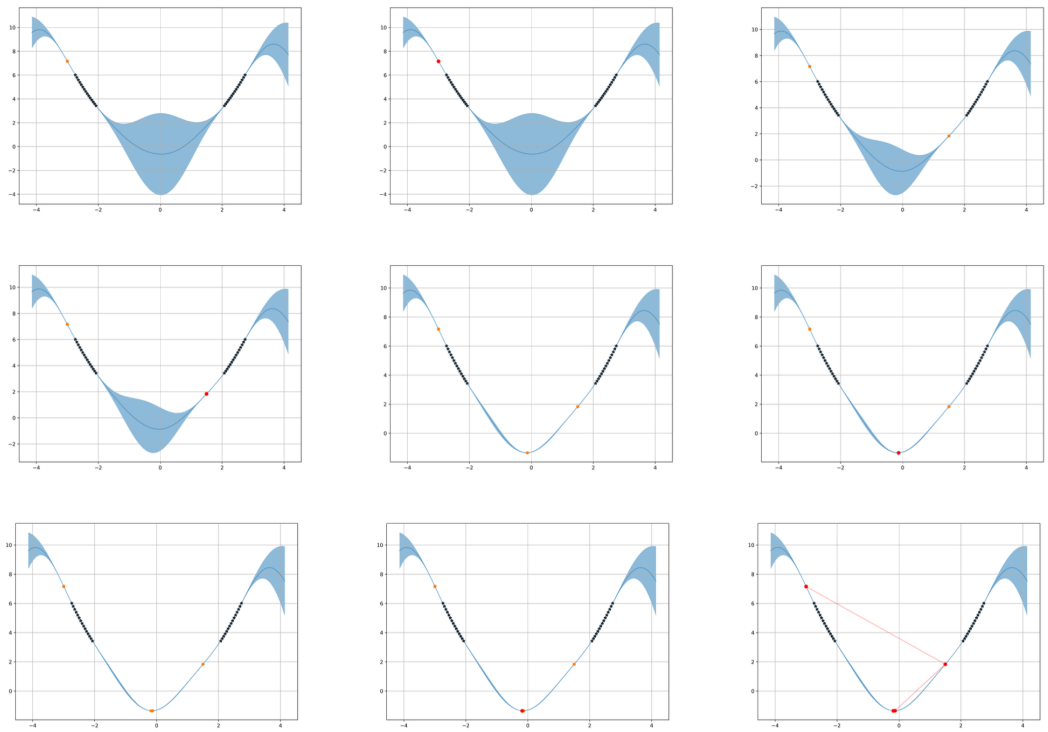

Figure 3: The process of Thompson sampling using L-BFGS finding the local minimum in a GP trained convex function.

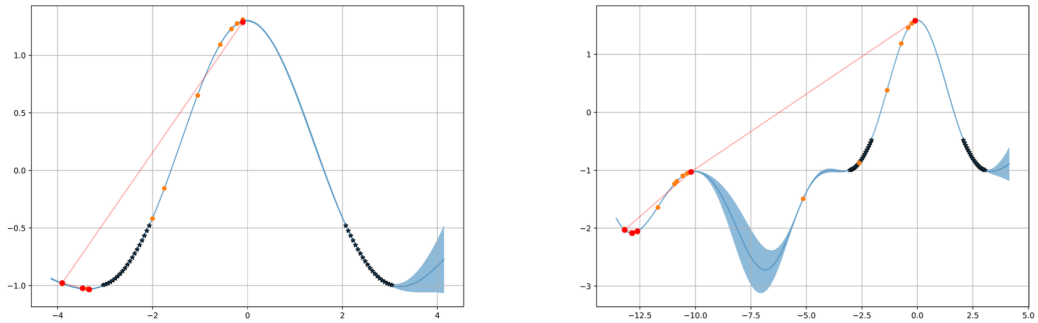

Figure 4: Comparison of Thompson sampling using L-BFGS finding the local minimum in a GP trained cosine function and when overshooting in the same function.

3.3 The 10-dimensional Ackley function

We examine performance on the 10-dimensional Ackley function in the domain $[5, 10]^{20}$. We compare Thompson sampling using L-BFGS as an acquisition function in vanilla BO and TuRBO from [4] in 10-dimensional Ackley function. Figure 6 shows the result for a total of 250 function evaluations, batch size q = 10, and 20 initial points for all algorithm. After observing L-BFGS overshooting when sampling, we chose to only try it in small optimization function. As a result, our method does not perform better than TuRBO.

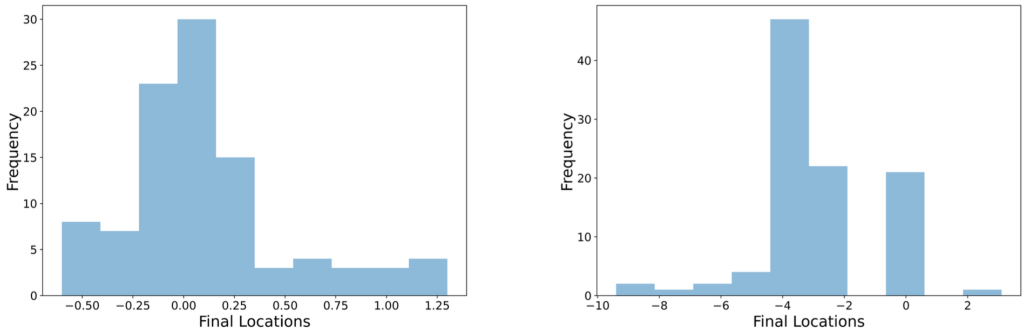

Figure 5: Histograms representing the final locations of Thompson sampling using L-BFGS in convex function and cosine function. For each histogram, the experiment is conducted 100 times.

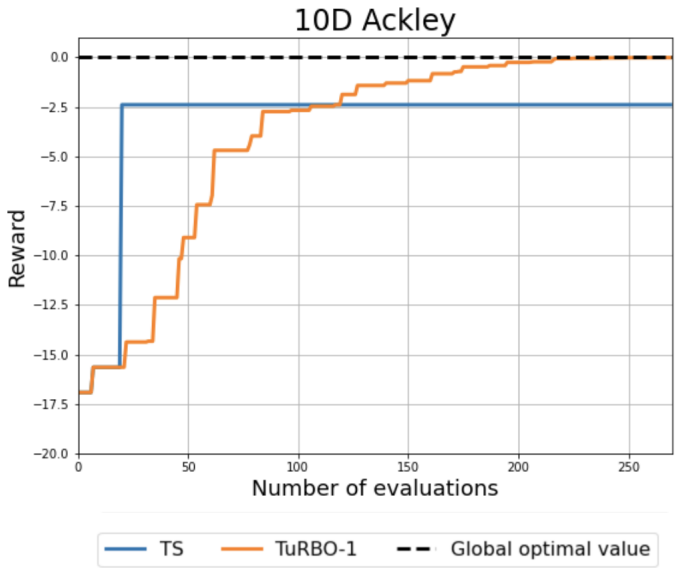

Figure 6: Comparison of Thompson sampling using L-BFGS in vanilla BO and TuRBO in 10-dimensional Ackley function.

4 Conclusions

We proposed Thompson sampling using derivative methods as a new acquisition function. We tried three methods, and among those methods, L-BFGS was the fastest in terms of finding the next sampling point in high dimension. Although L-BFGS is powerful when finding the local optimum, in some cases, it overshoots while doing line search like in Fgirue 4.

In the future, we plan to implement gradient descent with line search that we can control line search unlike L-BFGS and apply it as a new derivative method into Thompson sampling in order to make its efficiency stable and robust. Then, we intend to test it in vanilla BO in more complicated optimization functions such as 200-dimensional Ackley function and 60-dimensional Rover trajectory planning function [20]. Finally, we will apply it to TuRBO [4] as a new acquisition function and compare with original TuRBO. We hope our new acquisition function applied in TuRBO outperforms the original TuRBO and achieve the state-of-the-art Bayesian local optimization, particularly in the high-dimensional setting.

References

[1] Maximilian Balandat, Brian Karrer, Daniel R. Jiang, Samuel Daulton, Benjamin Letham, Andrew Gordon Wilson, and Eytan Bakshy. BoTorch: A Framework for Efficient Monte-Carlo Bayesian Optimization. In *Advances in Neural Information Processing Systems 33*, 2020.

[2] Naijia Anna Dong, David J. Eckman, Xueqi Zhao, Shane G. Henderson, and Matthias Poloczek. Empirically comparing the finite-time performance of simulation-optimization algorithms. In *2017 Winter Simulation Conference (WSC)*, pages 2206–2217, 2017.

[3] David Eriksson and Martin Jankowiak. High-dimensional bayesian optimization with sparse axis-aligned subspaces, 2021.

[4] David Eriksson, Michael Pearce, Jacob R Gardner, Ryan Turner, and Matthias Poloczek. Scalable global optimization via local bayesian optimization. *arXiv preprint arXiv:1910.01739*, 2019.

[5] David Eriksson and Matthias Poloczek. Scalable constrained bayesian optimization. In *International Conference on Artificial Intelligence and Statistics*, pages 730–738. PMLR, 2021.

[6] Peter I. Frazier. A tutorial on bayesian optimization, 2018.

[7] Jacob Gardner, Chuan Guo, Kilian Weinberger, Roman Garnett, and Roger Grosse. Discovering and exploiting additive structure for bayesian optimization. In *Artificial Intelligence and Statistics*, pages 1311–1319. PMLR, 2017.

[8] Jacob R Gardner, Geoff Pleiss, David Bindel, Kilian Q Weinberger, and Andrew Gordon Wilson. Gpytorch: Blackbox matrix-matrix gaussian process inference with gpu acceleration. *arXiv preprint arXiv:1809.11165*, 2018.

[9] Eduardo C. Garrido-Merchán and Daniel Hernández-Lobato. Predictive entropy search for multi-objective bayesian optimization with constraints. *Neurocomputing*, 361:50–68, 2019.

[10] Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *International conference on learning and intelligent optimization*, pages 507–523. Springer, 2011.

[11] Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4):455–492, 1998.

[12] Benjamin Letham, Roberto Calandra, Akshara Rai, and Eytan Bakshy. Re-examining linear embeddings for high-dimensional bayesian optimization, 2020.

[13] Mojmir Mutny and Andreas Krause. Efficient high dimensional bayesian optimization with additivity and quadrature fourier features. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.

[14] Amin Nayebi, Alexander Munteanu, and Matthias Poloczek. A framework for bayesian optimization in embedded subspaces. In *International Conference on Machine Learning*, pages 4752–4761. PMLR, 2019.

[15] Biswajit Paria, Kirthevasan Kandasamy, and Barnabás Póczos. A flexible framework for multi-objective bayesian optimization using random scalarizations, 2019.

[16] Carl Edward Rasmussen. Gaussian processes in machine learning. In *Summer school on machine learning*, pages 63–71. Springer, 2003.

[17] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.

[18] Jasper Snoek, Oren Rippel, Kevin Swersky, Ryan Kiros, Nadathur Satish, Narayanan Sundaram, Md. Mostofa Ali Patwary, Prabhat, and Ryan P. Adams. Scalable bayesian optimization using deep neural networks, 2015.

[19] William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25:285–294, 1933.

[20] Zi Wang, Clement Gehring, Pushmeet Kohli, and Stefanie Jegelka. Batched large-scale bayesian optimization in high-dimensional spaces, 2018.

[21] Ziyu Wang, Frank Hutter, Masrour Zoghi, David Matheson, and Nando de Freitas. Bayesian optimization in a billion dimensions via random embeddings, 2016.

[22] Ciyou Zhu, Richard H. Byrd, Peihuang Lu, and Jorge Nocedal. Algorithm 778: L-bfgs-b: Fortran subroutines for large-scale bound-constrained optimization. *ACM Trans. Math. Softw.*, 23(4):550–560, dec 1997.